Procs and lambdas are the big ones to talk about here. We'll start with them and bring methods back into the picture later.

Proc Objects

Although most things in Ruby are objects, blocks are not. But why would you care about that? Imagine that you want to store a block and execute it later. To do that, you need an object.

To solve this problem, Ruby provides the standard library class Proc. A Proc is a block that has been turned into an object. You can create a Proc by passing the block to Proc.new. Later, you can evaluate the block-turned-object with Proc#call:

```
inc = Proc.new {|x| x + 1 }
# more code...
inc.call(2) # => 3
```

This technique is called a *Deferred Evaluation*.

Spell: Deferred Evaluation

There are a few more ways to create Procs in Ruby. Ruby provides two *Kernel Methods (32)* that convert a block to a Proc: lambda and proc. In a short while, you'll see that there are subtle differences between creating Procs with lambda and creating them in any other way, but in most cases you can just use whichever one you like best:

```
dec = lambda {|x| x - 1 }
dec.class # => Proc
dec.call(2) # => 1
```

Also, you can create a lambda with the so-called "stabby lambda" operator:

```
p = ->(x) { x + 1 }
```

Notice the little arrow. The previous code is the same as the following:

```
p = lambda {|x| x + 1 }
```

So far, you have seen not one, but four different ways to convert a block to a Proc. There is also a fifth way, which deserves its own section.

The & Operator

A block is like an additional, anonymous argument to a method. In most cases, you execute the block right there in the method, using yield. In two cases, yield is not enough:

- You want to pass the block to another method (or even another block).
- You want to convert the block to a Proc.